# Prerequisites and objectives

- Prerequisites are recursion, linked structures, stacks, and induction.
- Describe trees precisely. Trace traversals, BST updates, and heap operations while preserving their respective invariants.
- Distinguish an arbitrary binary tree, a binary search tree, and a binary heap by their requirements rather than appearance alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Mathematical definition and terminology

- An undirected tree is a connected acyclic simple graph. Every pair of vertices has exactly one simple path, and an n-vertex tree has n−1 edges.
- Choosing a root gives each nonroot node a unique parent. Its children are adjacent nodes one edge farther from the root.
- Depth counts edges from the root. A node's height counts edges to its deepest descendant leaf. Tree height is root height.
- A leaf has no children. A forest is a disjoint collection of trees. We assign the empty tree height −1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Left and right positions have meaning

- A binary-tree node has at most one left child and one right child. A lone left child and a lone right child give different structures.
- A binary tree of height $h \geq 0$ has at most $2^{h+1}-1$ nodes, since depth d has at most $2^d$ nodes.
- Here a full binary tree has exactly two children at every internal node. A complete tree fills every level before the last, then fills the last from left to right without gaps.
- A perfect tree has all leaves at the same depth and two children at every internal node. Fix these conventions before using terms that vary across texts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Linked and array representations

- Linked binary nodes store `{key,left,right}` and naturally accommodate irregular shapes.
- A complete tree fits a zero-based array. Node i has children `2i+1,2i+2` and parent `⌊(i−1)/2⌋`, with the parent formula restricted to i>0.
- Complete-position numbering of a chain of height h can require exponentially large indices. Array representation is not space-efficient for every binary tree.
- Check multiplication and addition for overflow before computing child indices, then check the index against the element count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three depth-first traversals

- Preorder visits root, left, right. Inorder visits left, root, right. Postorder visits left, right, root.
- Our original tree has root 8, left child 3 with children 1 and 6, and right child 12 with left child 10.
- Preorder is `8,3,1,6,12,10`. Inorder is `1,3,6,8,10,12`. Postorder is `1,6,3,10,12,8`.
- Applying the same rule recursively to each subtree proves by induction that each node is visited once, giving Θ(n) time.

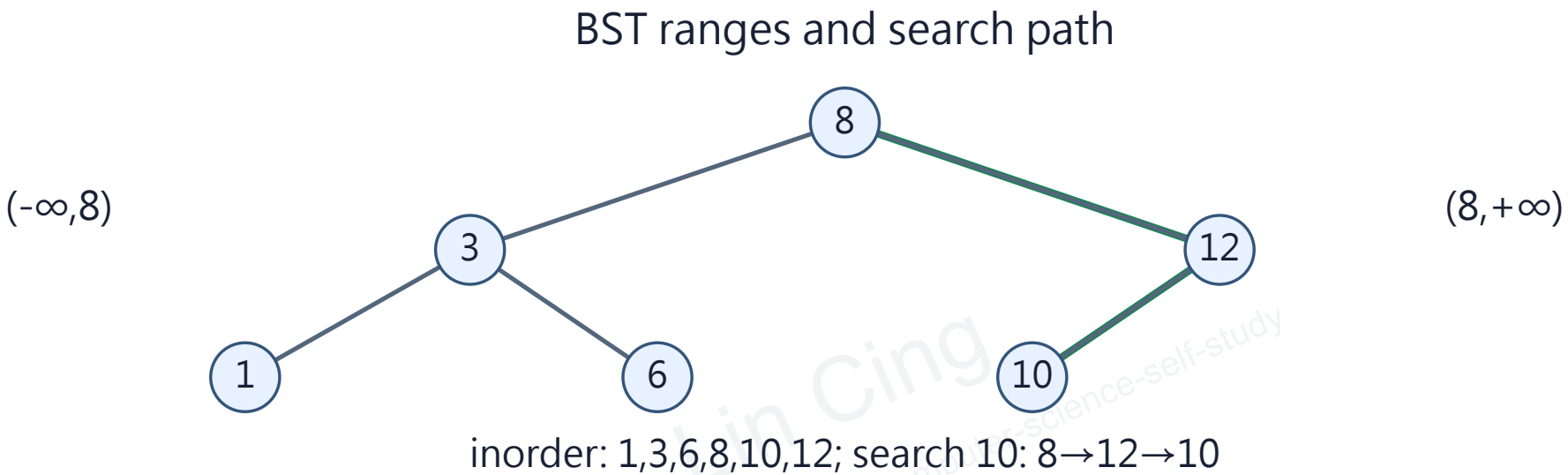
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Recursive traversals and complete iterative inorder states

```
visit(x,order):
    if x==null: return
    if order==pre: emit(x)
    visit(x.left,order)
    if order==in: emit(x)
    visit(x.right,order)
    if order==post: emit(x)
inorder_iterative(root):
    stack=[]; x=root
    while x!=null or stack is not empty:
        while x!=null: stack.push(x); x=x.left
        x=stack.pop(); emit(x); x=x.right
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Recursive traversals and complete iterative inorder states (continued)




Push 8,3,1 to obtain stack [8,3,1]. Pop 1 and emit [1]. Pop 3 and emit [1,3], then push 6 for stack [8,6]. Pop 6 and emit [1,3,6]. Pop 8, then follow 12's left chain to [12,10]. Pop 10 and 12, ending with [1,3,6,8,10,12]. The stack stores pending ancestors. Each node is pushed/popped once, giving Θ(n) time and O(h+1) space.

## Recursive traversals and complete iterative inorder states (continued)

Inorder of an arbitrary binary tree need not be sorted. That property requires the entire BST subtree-range invariant, not merely this example's output.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Stack and queue space in traversal

- Recursive DFS needs $O(h+1)$ extra stack space for a nonempty tree. A chain has $h=n-1$, so the bound is not always $O(\log n)$.
- Iterative inorder pushes the current left chain. It pops and outputs one node, then processes that node's right subtree.
- Level order uses a queue. The example gives `8,3,12,1,6,10`.
- Its queue needs $O(w)$ space, where $w$ is maximum level width. Even a balanced tree may have $w=\Theta(n)$, unlike its DFS stack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# BST invariant and search

- Keys are distinct. At every node x, all left-subtree keys are less than x.key and all right-subtree keys are greater.
- A comparison with k discards the root and one subtree. The whole-subtree invariant proves that the discarded region cannot contain k.
- Searching for 10 follows `8→12→10`. Searching for 11 ends at the empty right-child position of 10.
- Search, insertion, and deletion take O(h+1). Without balancing, increasing insertion order can produce linear height.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# BST insertion and duplicate policy

- Follow search to an empty child position. Adding a leaf there preserves every ancestor's allowed key interval.
- Inserting 11 into the example makes it the right child of 10. The path constrains `8<11<12` and `10<11`.
- A duplicate policy must agree with the invariant. Options include counts, unique composite keys, or explicit nonstrict subtree rules.
- In a key-value container, replacing an existing key's value differs from adding a node.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# BST lookup, insertion, and global-range validation

```
find(root,k):
    x=root
    while x!=null:
        if k==x.key: return x
        if k<x.key: x=x.left
        else: x=x.right
    return not_found
insert(x,k,v):
    if x==null: return Node(k,v,left=null,right=null)
    if k<x.key: x.left=insert(x.left,k,v)
    else if k>x.key: x.right=insert(x.right,k,v)
    else: x.value=v
    return x
valid(x,lower,upper):
    if x==null: return true
    return lower<x.key<upper and valid(x.left,lower,x.key)           and valid(x.right,x.key,upper)
root=insert(root,k,v)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## BST lookup, insertion, and global-range validation (continued)

An existing key updates its value without allocating another node. The validator starts with mathematical bounds $-\infty,+\infty$. An implementation over a bounded key type needs optional bounds; representable minimum and maximum keys cannot substitute for infinity.

Searching for 11 in the running tree compares $8<11,12>11,10<11$ and reaches the empty right child of 10. Insertion allocates there and returns the roots through 10.right,12.left,8.right and the external root. Ancestor intervals are $(8,+\infty),(8,12),(10,12)$, all containing 11. Reinserting 11 changes only its value.

In the counterexample root 8, left child 3, and right child 9 below 3, every local parent-child comparison passes. Global validation rejects 9 against inherited bounds $(3,8)$. Lookup correctness uses the same global invariant: if $k<x.key$, the entire right subtree has keys greater than x.key and cannot contain k. Lookup and insertion take $O(h+1)$ worst-case time; recursive insertion also uses $O(h+1)$ stack space. Validation examines every node and takes $\Theta(n)$, not $O(h)$.

## Three deletion cases

1. Remove a leaf from its parent link.
2. Replace a one-child node by its child subtree.
3. For two children, copy the logical contents of the minimum node in the right subtree, the inorder successor, then delete the successor's original position.

The successor has no left child, so its removal reduces to the first two cases. External node handles require a consistent choice between moving values and transplanting nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Worked two-child deletion trace

- Delete 8 from the original tree. The right-subtree minimum is 10. Change the root key to 10, then remove 12's left child 10.
- The new inorder is `1,3,6,10,12`, exactly the original sorted sequence without 8.
- The successor exceeds every key in the original left subtree and is less than every remaining right-subtree key. Replacement preserves order.
- Checking only parent-child comparisons is insufficient. A left child 3 of root 8 could have right descendant 9. Every local edge comparison holds, but the global BST invariant fails.

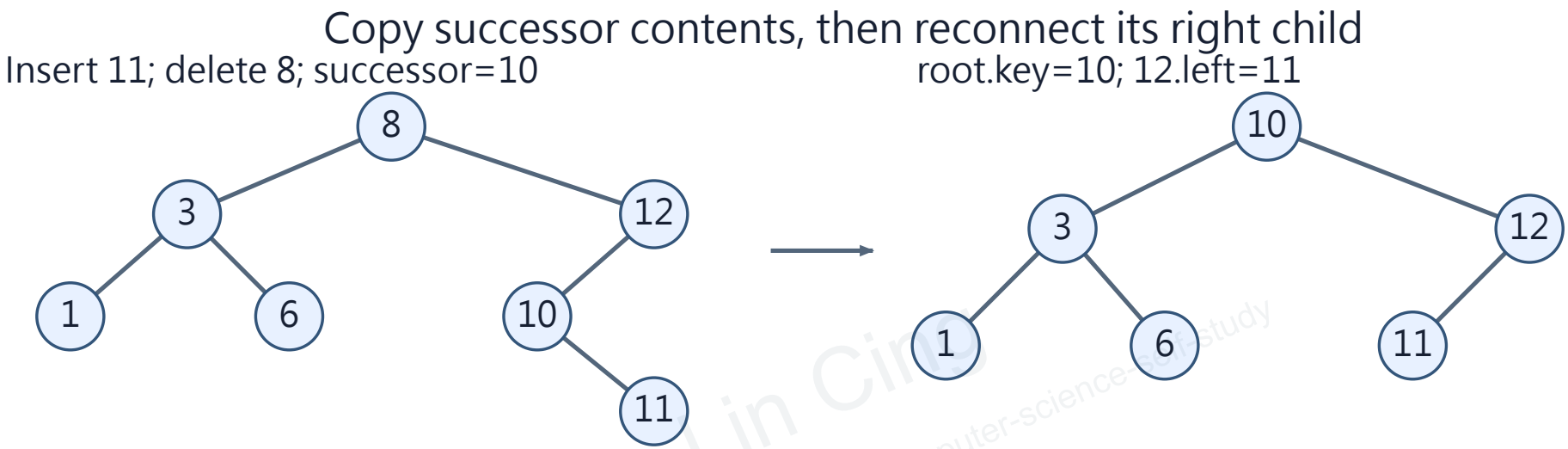
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Complete recursive BST deletion and identity contract

```
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x.left==null: y=x.right; release(x); return y
        if x.right==null: y=x.left; release(x); return y
        s=x.right
        while s.left!=null: s=s.left
        x.key_and_value=copy(s.key_and_value)
        x.right=erase(x.right,s.key)
    return x
root=erase(root,k)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Complete recursive BST deletion and identity contract (continued)




Insert 11 to the right of 10. Deleting 8 selects successor 10, which has no left child but has right child 11. After copying 10 to the root, recursive removal returns 11 into 12.left. Inorder loses exactly 8. The caller must assign the returned root; changing only a local pointer can leave an external root dangling.

## Complete recursive BST deletion and identity contract (continued)

This version preserves a key-value set, not node-handle identities. Stable handles to surviving elements require node transplantation with parent/root updates instead of content copying. Payload-copy exceptions need a separate commitment policy. Search plus the successor's left chain costs O(h+1), which can be Θ(n) without balancing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Ordinary successor: individual versus full-traversal cost

```
successor(x):
    require x is a live node and parent links are maintained
    if x.right!=null:
        x=x.right
        while x.left!=null: x=x.left
        return x
    p=x.parent
    while p!=null and x==p.right: x=p; p=p.parent
    return p
```

The successor is the smallest strictly greater key, not the numeric key plus one. If a right subtree exists, its leftmost node is the answer. Otherwise climb past ancestors whose right subtree has just been completed. The first ancestor containing the current subtree on its left is the answer; climbing beyond the root means the key is maximal. This representation stores ordinary children and parents, unlike the chapter’s tagged-thread representation.

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

## Ordinary successor: individual versus full-traversal cost (continued)

For root 11, left subtree 5(2,7), and right subtree 17(13,19), successor(5) follows the right child to 7. Successor(7) climbs through 5 to 11. Successor(19) climbs through 17 and 11 to null. Even with a node handle, one call may traverse height h and cost O(h+1), rather than O(1).

Starting at the minimum and repeatedly taking successors outputs 2,5,7,11,13,17,19. Each tree edge is traversed downward at most once and upward at most once, so initialization plus the full traversal costs Θ(n). Multiplying the maximum per-call cost by n gives only an upper bound, not a tight Θ(nh) result. Arbitrary disconnected successor queries cannot inherit the full-traversal average. Without parents, threads, or an explicit ancestor stack, the interface must search from the root again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Merging ordered sources and building a balanced tree linearly

```
merge_unique(streams):
    H=min_heap of each nonempty stream's first record; pending=none
    while H not empty:
        (key,value,source)=H.pop_min()
        if pending==none: pending=(key,value)
        else if key==pending.key: pending.value+=value
        else: emit pending; pending=(key,value)
        advance source; if nonempty: H.push(next record from source)
    if pending!=none: emit pending
build_sorted(count):
    if count==0: return null
    left_count=count//2
    left=build_sorted(left_count)
    (k,v)=next(unique_sorted_stream)
    x=Node(k,v,left=left,right=null,parent=null,size=count)
    if left!=null: left.parent=x
    x.right=build_sorted(count-left_count-1)
    if x.right!=null: x.right.parent=x
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Merging ordered sources and building a balanced tree linearly (continued)

Fix duplicate semantics first: let values count occurrences and sum equal-key values. Streams [2,7,17],[5,7,13],[11,19] contain eight records but seven unique keys 2,5,7,11,13,17,19; key 7 has value 2. With k sources and N input records, heap merging costs O(k+N log(k+1)), linear for fixed k. It is not O(N) for arbitrary growing k. The unique count U must be known: count and replay repeatable sources, or buffer U records for a one-pass source.

Build_sorted(7) first consumes three records into subtree 5(2,7), then consumes 11 as the root, then constructs 17(13,19). Each node consumes one record in inorder, so the resulting inorder equals the ordered input. Induction proves global BST ranges. Child counts differ by at most one, giving height O(log(U+1)). The recurrence $T(U) = T(\lfloor U/2 \rfloor) + T(U - \lfloor U/2 \rfloor - 1) + \Theta(1)$ gives $\Theta(U)$ construction time. Output nodes occupy Θ(U); the extra recursion stack uses O(log(U+1)).

Inserting U already sorted keys individually into an unbalanced BST instead creates a chain and takes 1+2+…+(U−1)=Θ($U^2$) comparisons. Sorted input helps direct construction but does not accelerate an ordinary insertion routine automatically. Taking array midpoints yields the same linear construction; this version demonstrates that random indexing is unnecessary.

# Threaded trees reuse empty pointers

- Tags distinguish child links from threads. Otherwise empty left and right positions can point to inorder predecessors and successors.
- For the inorder successor, follow a right thread directly. If the right link is a real child, enter that subtree and follow real left children to the end.
- A full traversal needs no recursion or explicit stack. Total time is O(n), with O(1) additional traversal state.
- Insertion and deletion must repair affected threads. Treating a thread as a recursive child can create nonterminating cycles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Concrete threaded traversal and insertion

- In the example, leaf 6 has predecessor thread 3 and successor thread 8. Leaf 10 has predecessor 8 and successor 12.
- After 6, follow its right thread to 8. Node 8 has a real right child, so enter 12 and follow its real left child to 10.
- Insert 11 to the right of 10 where a thread previously pointed to 12. Give 11 left and right threads to 10 and 12. Change 10's right link and tag to a real child pointing to 11.
- Inorder becomes `1,3,6,8,10,11,12`. Leaving 10's right tag as a thread makes search and traversal interpret the link incorrectly. Maintain pointer and tag as one update.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Inorder successor and right insertion with double threads

```
successor(x):
    if x.right_tag==thread: return x.right
    x=x.right
    while x.left_tag==child: x=x.left
    return x
insert_right(p,k):
    require p.right_tag==thread and k>p.key
    s=p.right
    require s==null or k<s.key
    x=Node(k,left=p,right=s,left_tag=thread,right_tag=thread)
    p.right=x; p.right_tag=child
    if s!=null and s.left_tag==thread and s.left==p: s.left=x
```

This variant has no header sentinel; extreme threads are null. Start at the leftmost real child and follow successors until null. Inserting11 at10's right-thread position sets new11.left=10 and 11.right=12 as threads, then10.right=11 as a child. If12.left is a real child pointing to 10, it must remain unchanged. Update it only if it was a predecessor thread to 10.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Inorder successor and right insertion with double threads (continued)

One successor may descend O(h), but a complete traversal uses real edges and threads only boundedly many times, giving O(n) total and O(1) extra traversal state. Pointer and tag jointly define representation. Updating one without the other can skip11 or produce cycles. Threaded deletion must repair these extra links in addition to ordinary BST order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Threaded-tree deletion, endpoints, and identity

Use the preceding doubly threaded representation: no header, null outer threads, and distinct keys. Search descends only through links tagged child. Reduce a two-child deletion to removing its successor with at most one real child, then repair neighboring threads and the parent link. Never recurse through a thread as though it were a subtree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Threaded-tree deletion, endpoints, and identity (continued)

```
predecessor(x):
    if x.left_tag==thread: return x.left
    x=x.left
    while x.right_tag==child: x=x.right
    return x
erase_threaded(k):
    p=null; z=root
    while z!=null and z.key!=k:
        if k<z.key:
            if z.left_tag==thread: return absent
            p=z; z=z.left
        else:
            if z.right_tag==thread: return absent
            p=z; z=z.right
    if z==null: return absent
    if z.left_tag==child and z.right_tag==child:
        p=z; s=z.right
        while s.left_tag==child: p=s; s=s.left
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

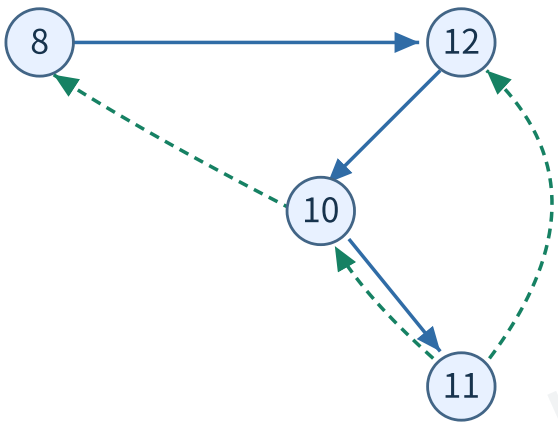
# Threaded-tree deletion, endpoints, and identity (continued)

```
        z.key_and_value=copy(s.key_and_value); z=s
    pred=predecessor(z); succ=successor(z)
    replacement=null
    if z.left_tag==child: replacement=z.left
    else if z.right_tag==child: replacement=z.right
    if pred!=null and pred.right_tag==thread and pred.right==z:
        pred.right=succ
    if succ!=null and succ.left_tag==thread and succ.left==z:
        succ.left=pred
    if p==null: root=replacement
    else if p.left_tag==child and p.left==z:
        if replacement!=null: p.left=replacement
        else: p.left=pred; p.left_tag=thread
    else:
        if replacement!=null: p.right=replacement
        else: p.right=succ; p.right_tag=thread
    release(z); return removed
```

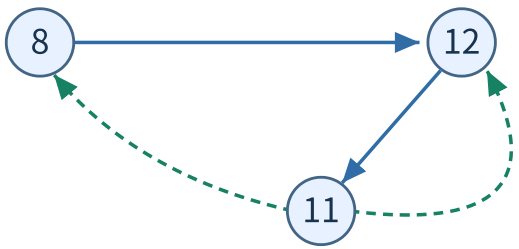
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Threaded-tree deletion, endpoints, and identity (continued)

Before: 12.left=10; 11.left(thread)=10




After: 12.left=11; 11.left(thread)=8




10: released

Blue solid: real child. Green dashed: thread. Null outer threads omitted.
Subtree {1,3,6} below 8.left is unchanged and omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Threaded-tree deletion, endpoints, and identity (continued)

Delete node 10 after it has right child 11 in the original example. Then p=12,pred=8,succ=11,replacement=11. Redirect 11's left thread from 10 to 8, redirect 12's real left child to 11, retain 11's right thread to 12, and only then release 10. Inorder changes from [1,3,6,8,10,11,12] to [1,3,6,8,11,12]. successor(8) descends through 12's real left child to 11; predecessor(11) follows its thread directly to 8.

Deleting leaf 6 changes parent 3's right child into a thread to successor 8; retaining its child tag is invalid. Deleting the singleton has pred=succ=replacement=null and sets root=null before release. Empty or unsuccessful deletion changes nothing. The two-child branch copies the complete key-value pair into the target and physically frees the successor. This version preserves dictionary contents, not undeleted node-handle identities; payload copy cost and exceptions are separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Threaded-tree deletion, endpoints, and identity (continued)

Correctness has three parts. The successor has no real left child, so the physical deletion has at most one child. Attaching replacement preserves all remaining real subtrees and inorder. Only the predecessor's successor thread, successor's predecessor thread, and parent's vacated-side tag can refer to the removed position; repairing them leaves no dangling thread. Update tags together with pointers and never overwrite a real child with a thread unconditionally. Search, successor reduction, and predecessor/successor lookup each follow at most a height-h path, for O(h+1) time and O(1) auxiliary pointers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A min-heap has shape and order invariants

- A binary min-heap is complete, and every parent key is at most each child key. Its root is the global minimum.
- A heap imposes neither left-child versus right-child order nor whole-left versus whole-right subtree order. BST search does not apply.
- `[2,5,4,9,7,8]` is a valid original heap, not a globally sorted array. Arbitrary-key search remains Θ(n) in the worst case.
- A Priority Queue ADT offers insert, find-min, and delete-min. A binary heap is one implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Sift-up and sift-down

- Insert 1 at the last position and swap upward while the parent is larger. It moves through indices 6,2,0, giving `[1,5,2,9,7,8,4]`.
- Delete-min replaces the root with the last element and swaps down with the smaller child until order is restored. An arbitrary child choice may leave a smaller child below its parent.
- Edges outside the update path are unchanged. Repairing each violated edge along the path preserves heap order and completeness.
- Heap restructuring for insert and delete-min takes worst-case O(log n). Find-min is O(1). Underlying-array growth adds allocation and movement costs, amortized across insertions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Complete binary min-heap sifting

```
insert(v):
    A.append(v); i=len(A)-1
    while i>0:
        p=(i-1)//2
        if A[p]<=A[i]: break
        swap(A[p],A[i]); i=p
delete_min():
    require len(A)>0
    result=A[0]; last=A.pop()
    if len(A)==0: return result
    A[0]=last; sift_down(0,len(A)); return result
sift_down(i,n):
    while i<n//2:
        j=2*i+1
        if j+1<n and A[j+1]<A[j]: j+=1
        if A[i]<=A[j]: break
        swap(A[i],A[j]); i=j
build_heap():
    for i=n//2-1 down to 0: sift_down(i,n)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete binary min-heap sifting (continued)

On [2,5,4,9,7,8], inserting 1 appends at index 6, swaps 6↔2 then 2↔0, and gives [1,5,2,9,7,8,4]. Delete-min replaces the root with final 4, giving [4,5,2,9,7,8]. Comparing children 5 and 2 selects 2, producing [2,5,4,9,7,8]. Child 8 is no smaller than 4, so sifting stops. A singleton removal returns before writing A[0].

The condition `i<n//2`, with integer division, means exactly that a left child exists. It avoids computing `2*i+1` before proving the index is in range. For n=3, only i=0 is internal; for n=4, both i=0 and i=1 are internal. Compute j only inside the loop and test j+1<n before reading a right child.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Why bottom-up construction is linear

- Sift down from the final internal node $\lfloor n/2 \rfloor - 1$ toward the root. Each processed node already has heap-ordered child subtrees.
- The number of nodes of height h is O(n/2^h), while a sift at height h costs O(h).
- Total work is $O(n \sum_{h \geq 0} h/2^h) = O(n)$. Reading the input takes Ω(n), so construction is Θ(n).
- Multiplying n nodes by an O(log n) maximum cost gives only a loose O(n log n) upper bound, not a tight Θ(n log n) result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Diagrams, experiments, and sources

- Default `std::priority_queue<int>` exposes the maximum. A min-heap requires an appropriate reversed comparator.
- The MIT 6.006 2020 index, L6, L8, supplies course context. The interface was checked against C++ priority_queue.
- Threaded trees follow the supplied topic scope. This chapter states its child/thread tags and sentinel conventions explicitly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study